

Doctor Visit Booking API

Техническа документация

REST API за управление на записвания на пациенти при лекари

FastAPI | SQLAlchemy 2.0 | SQLite | JWT Auth

Версия 0.1.0

Съдържание

1. Ръководство за използване на API
 - 1.1 Аутентикация
 - 1.2 Лекари
 - 1.3 Пациенти
 - 1.4 Работно време и графици
 - 1.5 Записвания (часове)
 - 1.6 Справочник на кодове за грешки
2. Описание на архитектурата
 - 2.1 Архитектура на високо ниво
 - 2.2 Структура на проекта
 - 2.3 Схема на базата данни (ER диаграма)
 - 2.4 Класова диаграма
 - 2.5 Диаграми на последователността
3. Самоанализ
 - 3.1 Спазване на SOLID принципите
 - 3.2 Проектни компромиси
 - 3.3 Области за подобрене

1. Ръководство за използване на API

API-то се сервира на `http://localhost:8000` по подразбиране. Интерактивен Swagger UI е наличен на `/docs`, а ReDoc на `/redoc`. Всички тела на заявки и отговори са в JSON формат. Защитените крайни точки изискват хедър `Authorization: Bearer <token>`.

1.1 Аутентикация

Системата използва JWT (JSON Web Tokens) за аутентикация. Токените се получават чрез крайната точка за вход и изтичат след 60 минути (конфигурируемо). Лекари и пациенти се регистрират отделно; и двата получават токен при успешна регистрация.

POST `/auth/register/doctor`

Регистриране на нов лекарски акаунт с начално работно време.

Заявка:

```
POST /auth/register/doctor
Content-Type: application/json

{
  "name": "Dr. Smith",
  "email": "smith@clinic.com",
  "password": "securepass123",
  "address": "123 Medical Ave",
  "working_hours": [
    {
      "day_of_week": 0,
      "start_time": "09:00:00",
      "end_time": "17:00:00",
      "is_break": false
    }
  ]
}
```

Отговор (201 Created):

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIs...\"",
  "token_type": "bearer"
}
```

POST `/auth/register/patient`

Регистриране на нов пациентски акаунт, свързан с личен лекар.

Заявка:

```
POST /auth/register/patient
Content-Type: application/json

{
  "name": "Jane Doe",
  "email": "jane@example.com",
  "password": "securepass123",
}
```

```
"phone": "+359888000000",
"doctor_id": 1
}
```

Отговор (201 Created):

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIs... ",
  "token_type": "bearer"
}
```

POST /auth/login

Аутентикация с имейл и парола.

Заявка:

```
POST /auth/login
Content-Type: application/json

{
  "email": "smith@clinic.com",
  "password": "securepass123"
}
```

Отговор (200 OK):

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIs... ",
  "token_type": "bearer"
}
```

Грешка (401 Unauthorized):

```
{
  "detail": {
    "code": "INVALID_CREDENTIALS",
    "message": "Incorrect email or password."
  }
}
```

1.2 Лекари

GET /doctors

Списък на всички регистрирани лекари (публичен, с пагинация).

Параметри: skip (по подразбиране 0), limit (по подразбиране 20, макс. 100)

Отговор (200 OK):

```
{
  "items": [
    {
      "id": 1,
      "name": "Dr. Smith",
      "email": "smith@clinic.com",
      "address": "123 Medical Ave",
      "working_hours": [
        {
          "id": 1,
          "day_of_week": 0,

```

```

        "start_time": "09:00:00",
        "end_time": "17:00:00",
        "is_break": false
      }
    ]
  },
  "total": 1,
  "skip": 0,
  "limit": 20
}

```

GET /doctors/{id}

Извличане на профила и работното време на конкретен лекар.

Отговор (200 OK): Същата структура като един елемент от горния списък.

Грешка (404 Not Found):

```

{
  "detail": {
    "code": "DOCTOR_NOT_FOUND",
    "message": "No doctor found with the given id."
  }
}

```

1.3 Пациенти

GET /patients/me

Извличане на профила на аутентикирания пациент. Изисква роля на пациент.

Хедъри: Authorization: Bearer <token>

Отговор (200 OK):

```

{
  "id": 2,
  "name": "Jane Doe",
  "email": "jane@example.com",
  "phone": "+359888000000",
  "doctor_id": 1
}

```

1.4 Работно време и графици

Лекарите управляват три нива на графични данни. При определяне на ефективния график за дадена дата системата проверява в ред: (1) активно временно предефиниране, покриващо датата; (2) последна постоянна промяна с `effective_date <= датата`; (3) базово работно време. Първото съвпадение печели.

PUT /doctors/me/schedule

Замяна на базовото работно време на лекаря (само за лекари).

Заявка:

```
PUT /doctors/me/schedule
Authorization: Bearer <doctor_token>

{
  "slots": [
    { "day_of_week": 0, "start_time": "09:00:00",
      "end_time": "12:00:00", "is_break": false },
    { "day_of_week": 0, "start_time": "12:00:00",
      "end_time": "13:00:00", "is_break": true },
    { "day_of_week": 0, "start_time": "13:00:00",
      "end_time": "17:00:00", "is_break": false }
  ]
}
```

Връща 200 с актуализиран `WeeklyScheduleResponse`. Връща 422 `SCHEDULE_CONFLICTS_APPOINTMENT` ако съществуващи записвания биха попаднали извън новите часове.

GET /doctors/{id}/schedule?date=YYYY-MM-DD

Извличане на ефективния график на лекар за конкретна дата. По подразбиране е днешната дата. Връща списък от `TimeSlotResponse` обекти.

POST /doctors/me/schedule/temporary

Създаване на временно предефиниране на графика (макс. 1 на лекар).

Заявка:

```
{
  "start_datetime": "2025-06-01T00:00:00Z",
  "end_datetime": "2025-06-08T00:00:00Z",
  "schedule": [
    { "day_of_week": 0, "start_time": "10:00:00",
      "end_time": "14:00:00", "is_break": false }
  ]
}
```

Връща 201 с `TemporaryOverrideResponse`. Връща 409 `OVERRIDE_EXISTS` ако лекарят вече има едно. Връща 422 `SCHEDULE_CONFLICTS_APPOINTMENT` ако съществуващи записвания конфликтуват.

DELETE /doctors/me/schedule/temporary

Премахване на активното временно предефиниране. Връща 204 No Content.

POST /doctors/me/schedule/permanent

Планиране на постоянна смяна на работното време (`effective_date` $\geq$ днес + 7 дни).

Заявка:

```
{
  "effective_date": "2025-07-01",
  "schedule": [
    { "day_of_week": 0, "start_time": "08:00:00",
      "end_time": "16:00:00", "is_break": false }
  ]
}
```

Връща 201 с `PermanentChangeResponse`. Връща 422 `EFFECTIVE_DATE_TOO_SOON` ако датата е по-малко от 7 дни напред. Връща 422 `SCHEDULE_CONFLICTS_APPOINTMENT` ако съществуващи записвания конфликтуват. Чакащите постоянни промени се прилагат автоматично към базовото работно време при стартиране на приложението и преди всяко разрешаване на график.

1.5 Записвания (часове)

POST /appointments

Създаване на записване (само за пациенти). Прилагани бизнес правила:

- Лекарят трябва да е личният лекар на пациента (403 NOT_PERSONAL_DOCTOR)
- Началото трябва да е поне 24 часа в бъдещето (422 TOO_SOON)
- Начало и край трябва да са в един и същ календарен ден (422 INVALID_TIME_RANGE)
- Интервалът трябва да попада в работен слот без почивка (422 OUTSIDE_WORKING_HOURS)
- Без препокриване с други планирани записвания (409 APPOINTMENT_OVERLAP)

Заявка:

```
POST /appointments
Authorization: Bearer <patient_token>

{
  "doctor_id": 1,
  "start_datetime": "2025-06-15T10:00:00Z",
  "end_datetime": "2025-06-15T11:00:00Z"
}
```

Отговор (201 Created):

```
{
  "id": 1,
  "doctor_id": 1,
  "patient_id": 2,
  "start_datetime": "2025-06-15T10:00:00+00:00",
  "end_datetime": "2025-06-15T11:00:00+00:00",
  "status": "scheduled",
  "cancelled_by": null
}
```

DELETE /appointments/{id}

Отмяна на записване (пациент или лекар). Правила:

- Потребителят трябва да е лекарят или пациентът (403 NOT_APPOINTMENT_OWNER)
- Отмяна поне 12 часа преди началото (422 CANCELLATION_TOO_LATE)
- Не може да се отмени вече отменено записване (409 ALREADY_CANCELLED)

Връща 204 No Content при успех.

GET /appointments

Списък на собствените записвания на аутентикирания потребител. Лекарите виждат записванията, където са лекар; пациентите виждат своите.

Параметри:

- skip, limit — пагинация (по подразбиране: 0, 20)
- date_from, date_to — филтър по период (ISO формат)
- status — филтър по 'scheduled' или 'cancelled'

Отговор (200 OK):

```
{
```



```
"items": [ ... ],  
"total": 5,  
"skip": 0,  
"limit": 20  
}
```

1.6 Справочник на кодове за грешки

Всички грешки връщат единна JSON структура:

```
{
  "detail": {
    "code": "ERROR_CODE",
    "message": "Human-readable description."
  }
}
```

HTTP	Код	Описание
401	INVALID_CREDENTIALS	Грешен имейл или парола
401	INVALID_TOKEN	Липсващ, изтекъл или невалиден JWT
403	NOT_PERSONAL_DOCTOR	Лекарят не е личен лекар на пациента
403	NOT_APPOINTMENT_OWNER	Потребителят не е страна по записването
403	DOCTOR_REQUIRED	Крайната точка изисква роля на лекар
403	PATIENT_REQUIRED	Крайната точка изисква роля на пациент
404	DOCTOR_NOT_FOUND	Няма лекар с това ID
404	PATIENT_NOT_FOUND	Няма пациентски профил
404	APPOINTMENT_NOT_FOUND	Няма записване с това ID
404	OVERRIDE_NOT_FOUND	Няма активно временно предефиниране
409	EMAIL_EXISTS	Имейлът вече е регистриран
409	APPOINTMENT_OVERLAP	Записването се препокрива
409	OVERRIDE_EXISTS	Лекарят вече има предефиниране
409	ALREADY_CANCELLED	Записването вече е отменено
422	TOO_SOON	По-малко от 24ч до записването
422	CANCELLATION_TOO_LATE	По-малко от 12ч до записването
422	OUTSIDE_WORKING_HOURS	Записването е извън раб. време
422	INVALID_TIME_RANGE	Краят е преди началото или преминава ден
422	EFFECTIVE_DATE_TOO_SOON	Пост. промяна < 7 дни напред
422	SCHEDULE_CONFLICTS_APPOINTMENT	Графикът конфликтва със записвания
422	INVALID_OVERRIDE_WINDOW	Край на предефинирането е преди началото

Обобщение на HTTP статус кодове

Код	Значение	Използва се за
200	OK	Успешно GET, PUT
201	Created	Успешно POST, създаващо ресурс
204	No Content	Успешно DELETE
400	Bad Request	Невалиден JSON / липсващи полета (Pydantic)
401	Unauthorized	Липсващ или невалиден JWT
403	Forbidden	Грешна роля или не е собственикът на ресурса
404	Not Found	Ресурсът не съществува

409	Conflict	Дублиране / препокриване / вече съществува
422	Unprocessable Entity	Нарушено бизнес правило

2. Описание на архитектурата

2.1 Архитектура на високо ниво

Приложението следва слойна архитектура с ясно разделение между HTTP обработка, бизнес логика и персистентност на данните:

```
+-----+
| HTTP Client |
+-----+
      | JSON over HTTP
+-----V-----+
| FastAPI Routers | Tonki HTTP adapteri: parsirovat
| (auth, doctors, | zayavka, vikat servis, vrashtat
| patients,       | otgovor. Nyama biznes logika.
| schedules,      |
| appointments)   |
+-----+
      |
+-----V-----+
| Service Layer | Vsichki biznes pravila, validacii
| (**/service.py) | i orkestraciya zhiveyat tuk.
+-----+
      | Hvarlya AppException.
+-----V-----+
| SQLAlchemy ORM | Deklarativni modeli definirat
| (**/models.py) | tablici, relacii i ogranich.
+-----+
      |
+-----V-----+
| SQLite (async) | aiosqlite drayver; Alembic
+-----+
      | upravlyava migracii.
```

Kros-sechenie:

- Pydantic shemi (**/schemas.py) definirat API kontrakta
- Izklycheniya (common/exceptions.py)
- Globalni error handlers (common/error_handlers.py)
- JWT middleware (dependencies.py + auth/security.py)

2.2 Структура на проекта

Кодовата база използва модулна организация по функционалност. Всяка домейнна концепция (auth, doctors, patients, schedules, appointments) има собствен пакет с router, service, schemas и models. Споделените утилити са в common/.

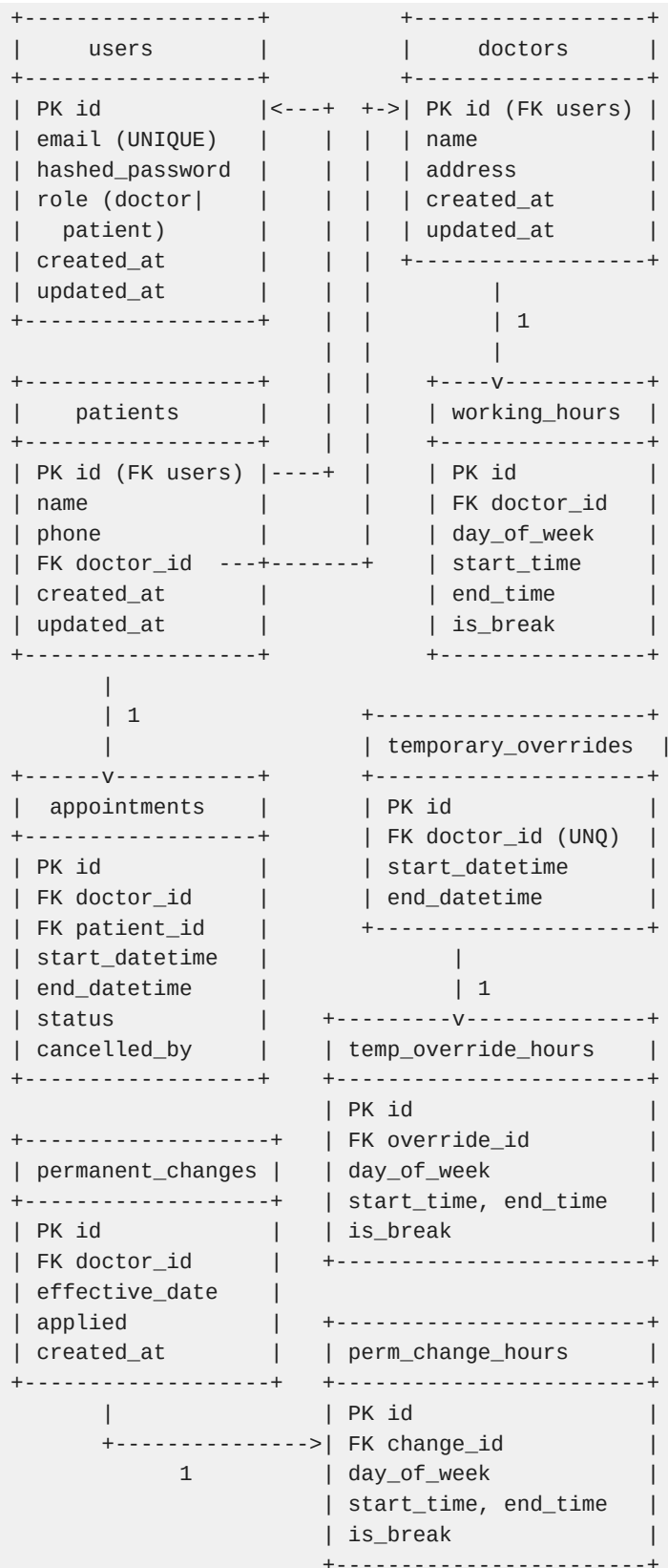
```
app/
+-- main.py           App factory, lifespan, router mounting
+-- config.py         Pydantic-settings (.env loading)
+-- database.py       Async engine, session factory, Base
+-- dependencies.py   get_db, get_current_user, role guards
+-- auth/            Registraciya, login, JWT
+-- doctors/         Spisak i detayli na lekari
+-- patients/        Profil na pacienta
+-- schedules/       Grafici, overrides, permanentni promeni
+-- appointments/    Zapisvaniya: sazdavane, otmyana, spisak
```

+- - common/

Izklyucheniya i error handlers

2.3 Схема на базата данни (ER диаграма)

Седем таблици с референтна цялост, осигурена чрез външни ключове. SQLite PRAGMA foreign_keys=ON се задава при всяка връзка.

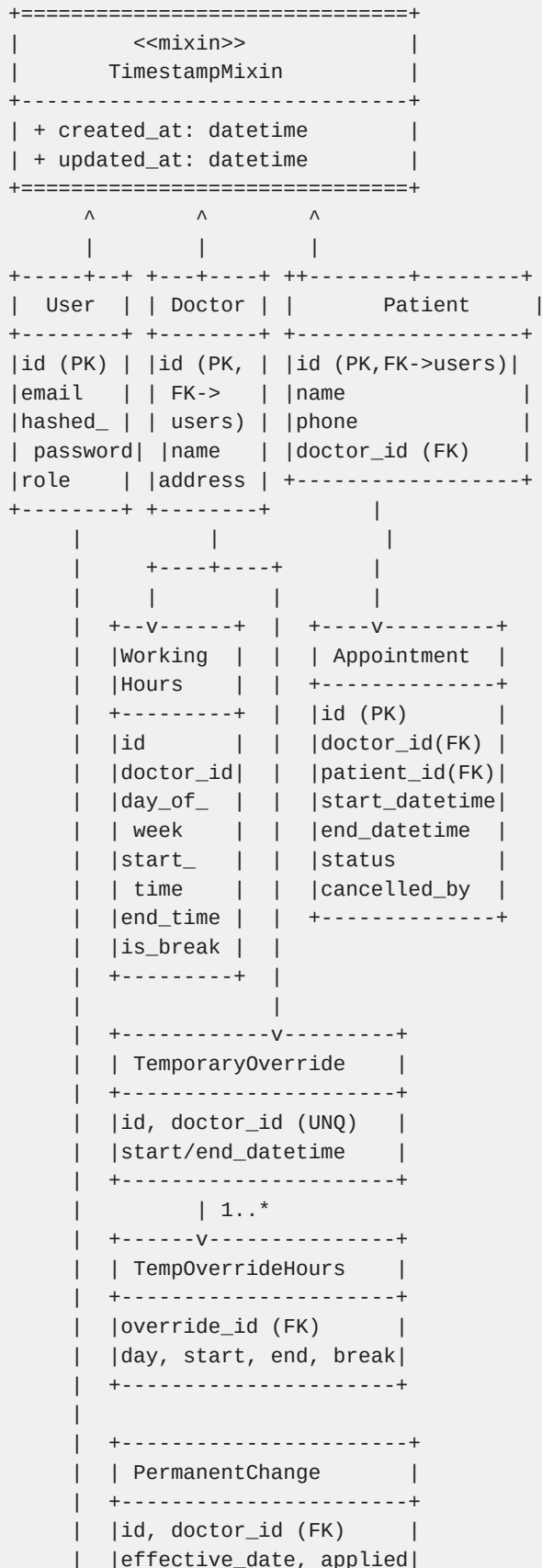


Ключови ограничения:

- `users.email` има UNIQUE индекс
- `temporary_overrides.doctor_id` е UNIQUE (макс. 1 на лекар)
- `working_hours` има UNIQUE(`doctor_id`, `day_of_week`, `start_time`)
- Записванията имат съставен индекс на (`doctor_id`, `start_datetime`, `end_datetime`)
- CHECK ограничения за валидни стойности на `role`, `status` и `day_of_week`

2.4 Класова диаграма (UML)

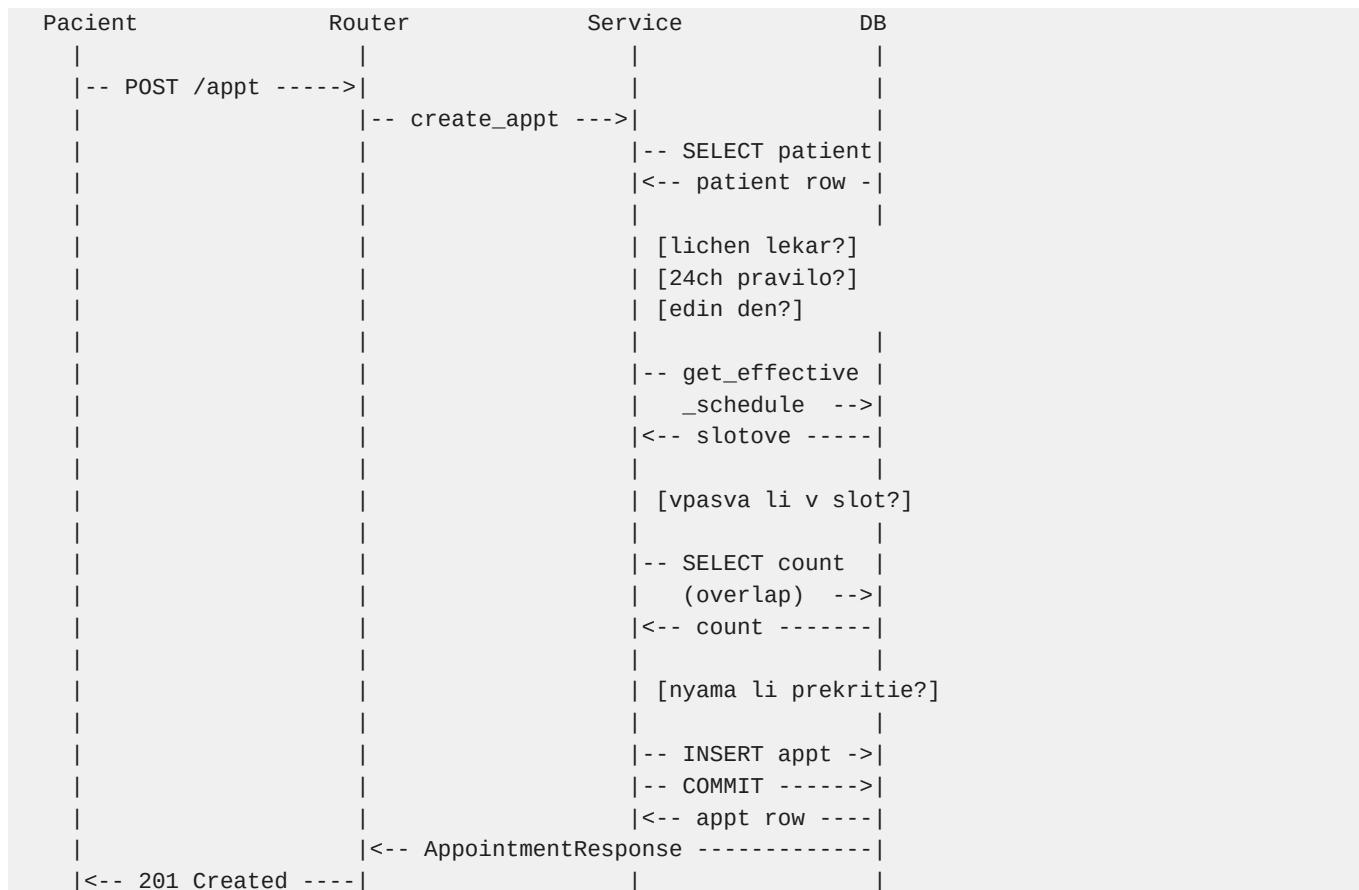
Класовата диаграма по-долу показва основните домейнни модели и техните връзки. Всички модели наследяват DeclarativeBase на SQLAlchemy. Моделите с времеви печати използват TimestampMixin.




```
| +-----+
| | 1..*
| +-----v-----+
| | PermanentChangeHours |
| +-----+
| |change_id (FK) |
| |day, start, end, break |
| +-----+
```

2.5 Диаграми на последователността

2.5.1 Поток на създаване на записване



2.5.2 Разрешаване на график (get_effective_schedule)



```
|
|
|      WHERE weekday ----->|
|      |<-- bazovi slotove -----|
|
|<-- list[TimeSlot] -|
```

2.5.3 Актуализация на график с проверка за конфликти

Lekar	Router	Service	DB
-- PUT schedule --->			
	-- update_wh ----->		
		-- DELETE old -->	
		-- INSERT new -->	
		-- FLUSH ----->	
		-- proverka za	
		konflikti --->	
		(za vsyako	
		planirano	
		zapisvane:	
		vpasva li?)	
	[ako konflikt: ROLLBACK, hvurlyane 422]		
	[ako OK: COMMIT]		
<-- 200 OK -----			

3. Самоанализ

3.1 Спазване на SOLID принципите

Принцип на единствената отговорност (SRP)

Всеки модул обработва точно една домейнна концепция. В рамките на модула отговорностите са допълнително разделени:

- Router-ите обработват само HTTP парсване и форматиране на отговорите.
- Service-ите съдържат цялата бизнес логика, валидации и оркестрация.
- Моделите дефинират персистентната структура и ограниченията в базата.
- Схемите дефинират API контракта (валидация на заявки, форма на отговори).
- Пакетът `common/` централизира крос-сеченията: класове за изключения в `exceptions.py` и глобални обработчици на грешки в `error_handlers.py`.

Пример: `appointments/service.py` притежава всички правила за записвания (24ч, препокриване, раб. време). Router-ът никога не проверява бизнес правила; делегира изцяло на `service-a`.

Принцип отвореност/затвореност (OCP)

Системата за разрешаване на графици демонстрира OCP. `get_effective_schedule()` проверява три слоя по приоритет (временно предефиниране > постоянна промяна > базово работно време). Добавянето на нов слой (напр. календар за празници) изисква само нов модел и още една стъпка на проверка, без промяна на съществуващата логика.

Йерархията на изключения (`AppException` -> `BusinessRuleException`, `ConflictException` и др.) също е отворена за разширение. Нови типове грешки могат да се добавят без промяна на глобалния `error handler`, който хваща базовия `AppException` клас.

Принцип на заместването на Лисков (LSP)

`Doctor` и `Patient` са свързани към `User` чрез 1:1 външен ключ. Всеки код, който работи с `User` (аутентикация, създаване на токен, `get_current_user`), работи еднакво независимо дали потребителят е лекар или пациент. Полето `role` се използва само за авторизационни проверки, но самият поток на аутентикация е напълно заместим.

Принцип на разделяне на интерфейсите (ISP)

Pydantic схемите са ограничени по операция. `DoctorRegisterRequest`, `PatientRegisterRequest`, `LoginRequest` и `TokenResponse` са всички отделни модели. Модулът за записвания има отделни `AppointmentCreateRequest`, `AppointmentResponse` и `AppointmentListFilters`. Никой клиент не е принуден да работи с полета, нерелевантни за неговата операция.

Зависимостите на FastAPI (`get_current_user`, `require_doctor`, `require_patient`) са фино гранулирани. Крайните точки декларират точно необходимото ниво на аутентикация.

Принцип на инверсия на зависимостите (DIP)

Service-ите зависят от абстракцията `AsyncSession`, никога от конкретни детайли на engine. Това се демонстрира от тестовете, които заменят производствената SQLite база с база в паметта чрез предефиниране на `get_db` зависимостта. Не са нужни промени в кода на service-ите.

Патърнът за dependency injection чрез `Depends()` механизма на FastAPI разделя обработката на заявки от управлението на DB сесии и аутентикацията.

3.2 Проектни компромиси

SQLite срещу PostgreSQL

SQLite е избран заради простотата (нулева конфигурация, файлово базиран). Това отговаря на обхвата на проекта, но ограничава едновременния запис и напредналите функции. Асинхронният драйвер (`aiosqlite`) обвива синхронния SQLite в thread pool. Миграция към PostgreSQL изисква само промяна на `DATABASE_URL` и инсталиране на `asyncpg`; всички заявки използват ORM и са независими от базата.

Незабавно промотиране на постоянни промени

Постоянните промени на графика се промотират към `working_hours` при стартиране (`lifespan`) и преди всяко извикване на `get_effective_schedule()`. Това е просто и коректно, но изпълнява глобално сканиране на всички неприложени промени при всяко четене на график. За проект в този мащаб това е незначително; при мащабиране би бил по-подходящ фонов планировчик (`Celery beat`, `APScheduler` или `cron`).

Проверка за конфликти при промяна на графика

Когато лекар актуализира графика си, системата проверява всички планирани записвания спрямо новите ефективни часове и отхвърля промяната ако някое записване би останало без покритие. Алтернативен подход би бил автоматична отмяна на конфликтующите записвания, но подходът с отхвърляне (422) е избран за безопасност: никой пациент не губи записване без явно действие.

JWT без refresh токени

Текущата система за аутентикация издава един access token с 60-минутен срок. Няма refresh token поток. Това опростява имплементацията, но означава, че клиентите трябва да се аутентикатират отново след изтичане. Добавянето на refresh токени би било естествено подобрение за производствена употреба.

3.3 Области за подобрение

- Пагинация: Използване на cursor-based пагинация вместо offset/limit за по-добра производителност при големи набори от данни.
- Rate limiting: Няма ограничение на честотата на заявките. Производствени деплойменти трябва да добавят middleware или throttling на ниво reverse proxy.
- Refresh токени: Имплементиране на поток за опресняване на токени за по-добро потребителско изживяване.
- Имейл известия: Уведомяване на пациенти и лекари при създаване, отмяна и промяна на график.
- Админ крайни точки: Няма админ роля. Добавянето ѝ би позволило централизирано управление.
- Soft-delete и одитен след: Отменените записвания използват флаг за статус, но другите ентитети се изтриват напълно. Одитен лог би подобрил проследимостта.
- Часови зони: Всички дати се съхраняват в UTC. Конвертирането се оставя на клиента. Явно

timezone-aware API би подобрило употребимостта.

- Фонова задача за промоции: Замяна на on-demand apply_pending_permanent_changes с фонов планировчик.
- Docker health checks: Dockerfile и docker-compose.yml нямат HEALTHCHECK директива.
- API версионирание: Всички маршрути са без версия. Префикс /v1/ би позволил обратно-съвместима еволюция.